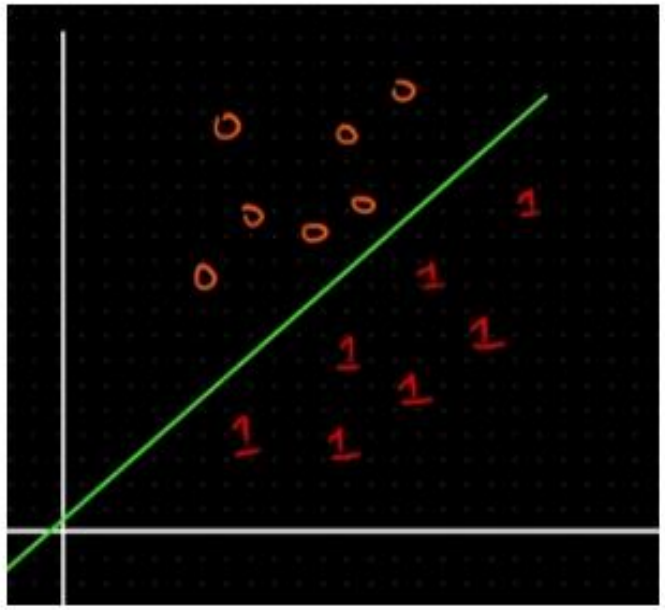
Logistic Regression

- Imagine you are at a crossroads where each path leads to a distinct outcome, and your choice is binary: yes or no, true or false, A or B.
- At its core, Logistic Regression is about probabilities. It measures the likelihood of an event occurring.
- It is a supervised machine learning algorithm to perform classification and works best for binary classification.

Healthcare Problem

Given features (BP, Weight, Blood count------)

Targeted outcome—Diabetic or not (0 or 1) –Binary classification

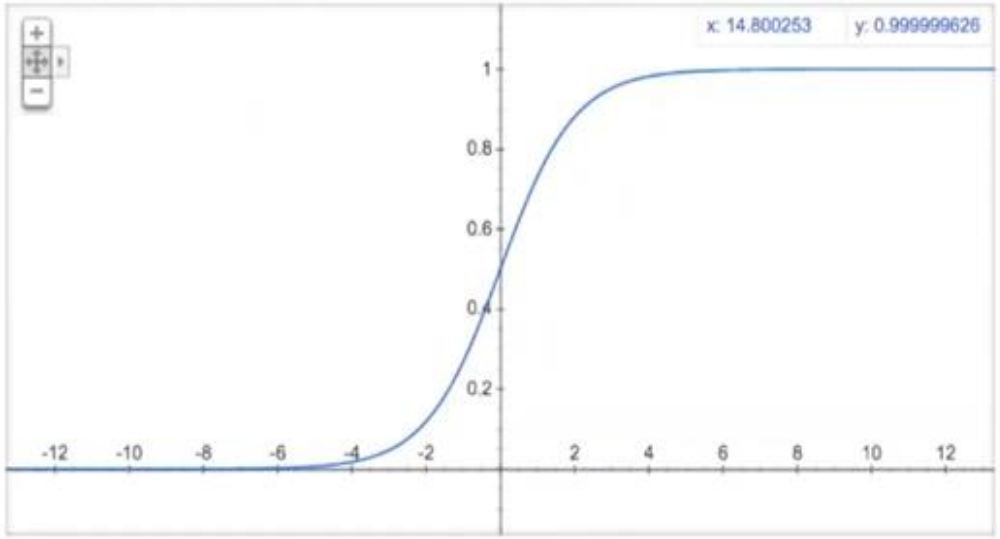


Sigmoid Function

Sigmoid Function

F(x)=1/1+e^-x

Graph for 1/(1+e^-x)



- Graphically we need to find a line or plane that best divides both the classes with minimum error
- **Equation of line y=mx+c Equation of plane w^x +c=0**
- Yi w^ xi> 0 (Correctly classified)
- We need to find a plane where Summation of Yi w^ xi> 0 from i to n is maximum
- (xi, yi) – fixed from test Data Set Dn
- Variable=w (Find a optimal value of w to maximize the summation of given equation)
- But this equation solely cannot be trusted upon as it depreciates in case of outliers.
- We need to find a function which doesn't allow the distance of a particular point to exceed more than a confined value and tapper off above that value.
- This method is also called Squashing because we are squeezing large distances between 0 to 1 where x varies linearly for smaller values.
- So the equation becomes summation of arg max of w from I to n f(yi w^T xi)
- Where f(sigmoid func) = 1/1+e^-x

Monotonic Function

- In **logistic regression**, *monotonic functions—especially the logarithm—are used to make optimization easier while preserving the same optimal weights w*
- The logarithm is a strictly monotonic increasing function:

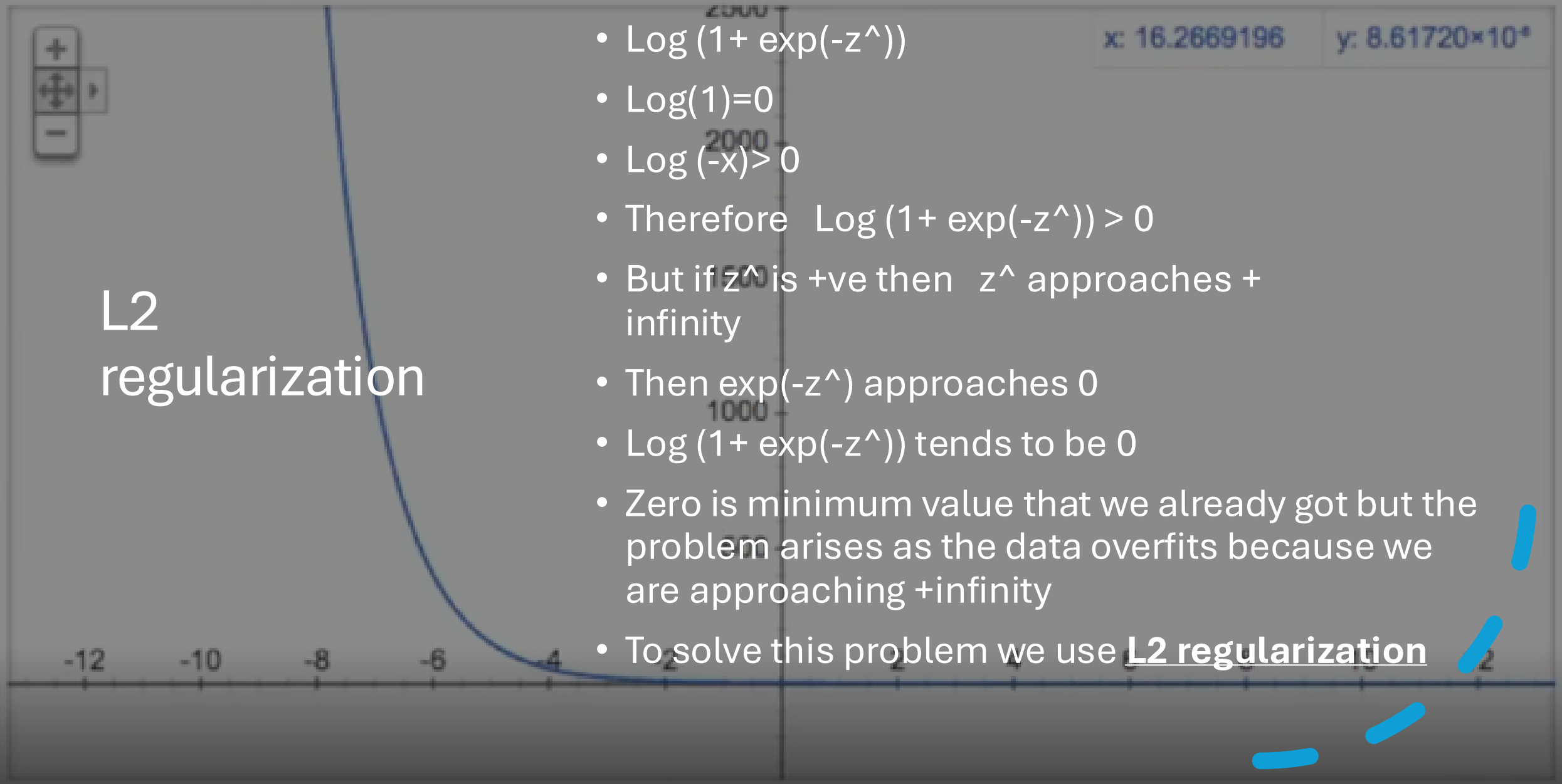
$$\log(a) > \log(b) \quad \text{iff} \quad a > b$$

$$w^* = \arg\max_{w} \sum_{i=1}^{n} \frac{1}{1+\exp(-y_i w^T x_i)}$$

Therefore the above equation changes to:

$$w^* = \arg\min_{w} \sum_{i=1}^{n} \log\left(1+\exp(-y_i w^T x_i)\right)$$

Graph for exp(-x)



L2 regularization

- Z^ = (yi wT xi)
- Log (1+ exp(-z^))
- Log(1)=0
- Log (-x)> 0
- Therefore Log (1+ exp(-z^)) > 0
- But if z^ is +ve then z^ approaches + infinity
- Then exp(-z^) approaches 0
- Log (1+ exp(-z^)) tends to be 0
- Zero is minimum value that we already got but the problem arises as the data overfits because we are approaching +infinity
- To solve this problem we use **L2 regularization**

L2 Regularization

$$w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} \log\left(1+\exp(-y_i w^T x_i)\right) + \lambda w^T w$$

$$\lambda w^T w = \lambda \|w\|_2^2$$

square of L_2 norm of w

$$= \lambda \sum_{j=1}^{d} w_j^2$$

prevents overfitting by adding a penalty term (sum of squared weights) to the cost function, shrinking coefficients towards zero without making them exactly zero, leading to a simpler, more generalized model

When Lambda = 0 we overfits to the training data

Lambda is very large then we underfit as the impact of cost function almost becomes negligible.

Loss Minimization

$$W^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} \log(1 + \exp(-y_i W^T x_i)) \text{ where } y_i \,\varepsilon\, \{=+1, -1\}$$

$$W^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} \log(1 + \exp(-Z_i))$$

where $Z_i = y_i W^T x_i$ *or can also be written as*

$Z_i = y_i f(x_i)$ *where* $f(x_i) = W^T x_i$

$f(x_i)$ *can be any function, in Logisitic regression this happens to be* $W^T x_i$, *which is the function that determines whether* x_i *belongs to the* $+/-$ *ve class*

Loss Minimization

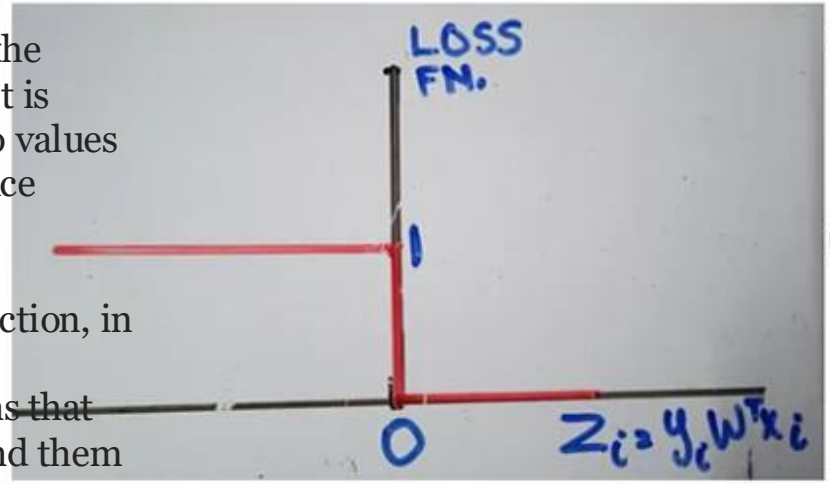
- An ideal loss function returns a value
- +1 for incorrectly classified points
- 0 for correctly classified points

$$W^* = \underset{w}{\text{argmin}} \underbrace{\begin{pmatrix} number\ of\ incorrectly \\ classified\ points \end{pmatrix}}_{\substack{this\ is\ the\ loss\ term\ that \\ we\ want\ to\ minimize}}$$

$$Z_i = y_i W^T x_i \Longrightarrow will\ produce\ 0\ for\ correctly\ classified\ points$$

$$\Longrightarrow will\ produce\ 1\ for\ in-correctly\ classified\ points$$

To solve optimization problems in ML, we need to use differentiation /calculus. The above 01loss function is not differentiable. For a function to be differentiable it has to be continous. The 01loss function is not differentiable at 0,which means at Zi equals 0 the value of the loss function is ambigious.From the above graph it is evident that when Zi is 0 , the loss function can take of the two values 0 or 1, which makes the loss function non-continuous and hence becomes non-differentiable.

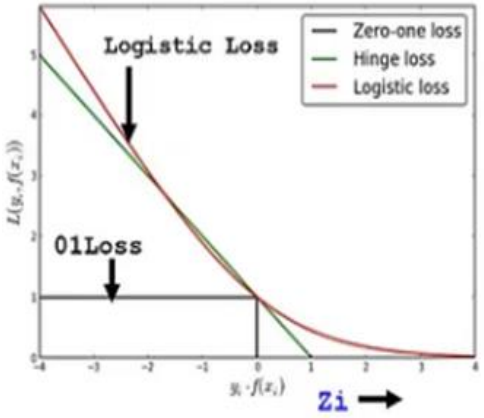


$$W^* = \underset{w}{\text{argmin}}\ 01Loss(x_i, y_i, W_i)$$

$$01Loss(Z_i) = \begin{cases} 1\ if\ Z_i < 0 \\ 0\ if\ Z_i > 0 \end{cases}$$

The solution to the above problem is we will approximate the function, in our ase this is the 01loss function . One such approximation is the **LOGISTIC LOSS** . There are thousands of other functions that can mimic the logistic loss function provided we are able to find them easily.

Logistic Loss



$$\text{Logistic loss} = \log(1 + \exp(-y_i * f(x_i)))$$

where $f(x_i) = W^T x_i$ and $Z_i = y_i * f(x_i)$

$$\text{Logistic loss} = \log(1 + \exp(-Z_i))$$

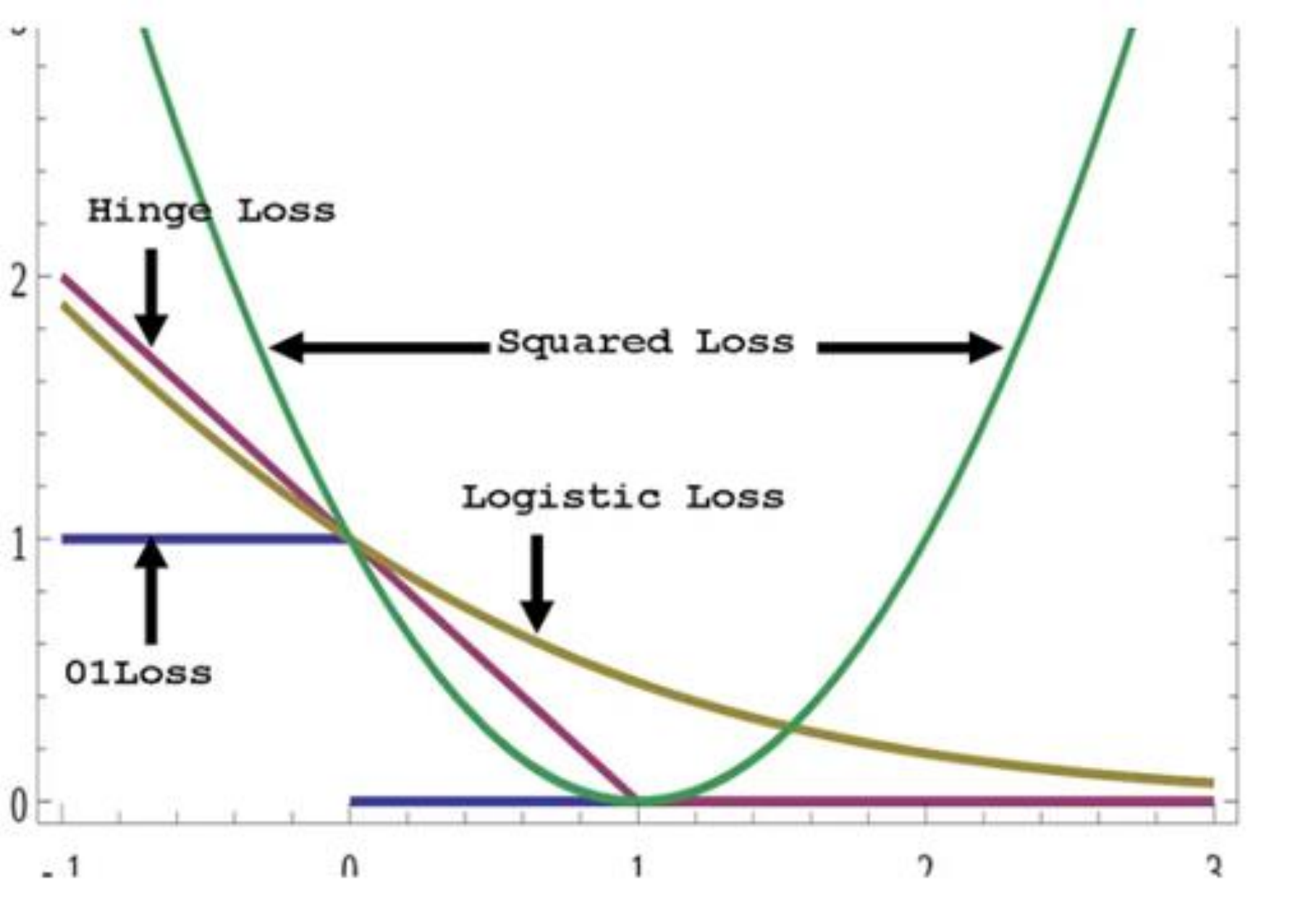
When Z_i takes small positive values the loss equals some small positive value

As Z_i increases the logistic loss will quickly tend towards 0

If Z_i moves to the negative side the loss increases dramatically

From the Loss minimization perspective/interpretation, if we slightly change the form of the loss function we get other machine learning classification algorithms.

Loss	Algorithm
Logistic Loss	Logistic Regression
Hinge Loss	Support Vector Machines
Exponential Loss	Ada Boost
Squared Loss	Linear Regression


Hinge Loss
Squared Loss
Logistic Loss
01Loss
2
1
0
-1
0
1
2
3